ORCA — KT Appendix A: File Purposes & Data Flow Traces

> Companion to the main KT document. This appendix does two things:
>
> 1. **Part 1** — every code file: what it does, why it matters, its role in ORCA, and an importance rating.
> 2. **Part 2** — three concrete end-to-end traces: you click **Analyse**, and we follow the data through every file, function, and decision until the result lands back on your screen.
>
> Read Part 1 to know *what each piece is*. Read Part 2 to see *how they work together*. Part 2 is where it truly clicks.

PART 1 — File-by-file purpose & importance

Importance legend: ⭐⭐⭐ = core, you must understand it · ⭐⭐ = important · ⭐ = supporting/reference

---

Data & Database layer

`data/` (synthetic data generation scripts)

- **What it does:** Generates the fake-but-realistic retail dataset — stores, SKUs, suppliers, capital pools, events, and inventory positions — and writes them into the SQLite database in three layers (raw → staged → curated).
- **Why it matters:** Without it there's no data for anything to run on. It's the "factory" that creates ORCA's world.
- **Role in ORCA:** One-time setup. Models the real UAE retail dataset from the original Palantir RCC project, but synthetic so it's shareable.
- **Importance:** ⭐ (run once, rarely touched)

`data/scheduler.py`

- **What it does:** Simulates the real-time monitoring that, in a live system, would run continuously. Scans inventory positions and marks SKUs as `CRITICAL` or `AT_RISK` in the `alerts` table. Run with `--once` for a single pass.
- **Why it matters:** It's what populates the alert list the dashboard shows. No scheduler run = empty Command Centre.
- **Role in ORCA:** The "trigger" source. In Palantir, the Ontology auto-fired when status changed; here `scheduler.py` plays that role.
- **Importance:** ⭐⭐

`db/queries.py`

- **What it does:** Every raw database read/write as a plain Python function (SQL → dict). E.g. `get_all_positions_for_sku()`, `get_supplier_for_sku()`, `get_capital_pool()`, `writeback_reorder_for_all_positions()`.
- **Why it matters:** It's the single, clean data-access layer. Nothing else writes SQL. This is the **open-source replacement for Palantir's Ontology object queries**.
- **Role in ORCA:** The foundation every tool and agent ultimately depends on for facts.
- **Importance:** ⭐⭐⭐

`db/pipeline_log.py`

- **What it does:** Creates and writes to the `pipeline_log` audit table — saves every completed pipeline run (status, all agent outputs, briefing).
- **Why it matters:** Audit trail. You can look back at what every pipeline decided and why. Also feeds the dashboard's pipeline history.
- **Role in ORCA:** Persistence + auditability (replaces Palantir's writeback object for state).
- **Importance:** ⭐⭐

`db/orca.db`

- **What it does:** The actual SQLite database file holding all tables.
- **Why it matters:** It's the single source of truth for structured facts.
- **Role in ORCA:** The database. Auto-created on first run; rebuilt by the data scripts.
- **Importance:** ⭐⭐⭐ (it's the data — but you don't edit it by hand)

Agent / AI layer

`agents/graph.py` — THE CORE FILE

- **What it does:** Defines the entire LangGraph pipeline: the shared `AgentState`, the four agent nodes (`agent1_node` … `agent3_node`), the pure-Python `route_node`, the three terminal nodes (`hitl_node`, `execute_node`, `suspend_node`), the `save_node`, all the edges wiring them together, and the HITL interrupt. Exposes `run_pipeline()`, `resume_pipeline()`, and `get_pipeline_state()`.
- **Why it matters:** This is the orchestrator. Everything else is a dependency this file pulls in. If you read one file end to end, read this one.
- **Role in ORCA:** The brain stem — turns a single SKU trigger into a full reorder decision.
- **Importance:** ⭐⭐⭐
- **Key internals to know:**
  - `AgentState` (TypedDict) — the shared "notebook" carrying every agent's inputs and outputs down

the chain.

- `interrupt_before=["execute_node"]` — the HITL pause.
- `MemorySaver` checkpointer keyed by `thread_id = pipeline_id` — how a paused pipeline resumes.
- It initialises the RAG retriever and is where the `C:/lit` path line lives (Issue #12).

`agents/llm_factory.py`

- **What it does:** One function, `get_llm()`, returns the correct LLM object based on `LLM_PROVIDER` in `.env` (groq / openai / ollama). Plus `get_provider_name()` and `get_model_name()` for logging.
- **Why it matters:** Single switch point. Change one line in `.env` and the whole system swaps LLM provider. **You will reuse this in your judge eval** (point it at `llama-3.3-70b-versatile`).
- **Role in ORCA:** Provider abstraction — proves ORCA isn't locked to one vendor (a real interview question).
- **Importance:** ⭐⭐⭐ (especially for you — eval reuses it)

`agents/prompts.py`

- **What it does:** Holds the four agents' system prompts as LangChain `ChatPromptTemplate` objects with `{placeholders}`. Exposes a `PROMPTS` dict (`PROMPTS["agent1"]` etc.).
- **Why it matters:** This is where the *business rules in natural language* live — the actual instructions that make each agent behave. These were translated directly from the Palantir RCC agent prompts.
- **Role in ORCA:** The "personality and rules" of each agent. `graph.py` fills the placeholders with real data and sends them to the LLM.
- **Importance:** ⭐⭐⭐ (the agent behaviour you evaluate is shaped here)

`agents/tools.py`

- **What it does:** The six `@tool`-decorated functions the agents use to fetch structured data. Each wraps one or more `queries.py` functions and sometimes adds derived fields.
- **Why it matters:** These are the agent's "hands" — how it reaches into the database.
- **Role in ORCA:** The tool layer. Also exposed via MCP (see `server.py`).
- **Importance:** ⭐⭐⭐
- **The six tools (and what each returns):**

Tool	Wraps (queries.py)	Returns / adds
`check_inventory_positions(sku_id)`	`get_all_positions_for_sku()`	positions + `critical_count`, `at_risk_count`, `total_current_stock`, `total_projected_shortfall`
`get_sku_info(sku_id)`	`get_sku_details()`	SKU master (category, abc_class, margin)

`get_supplier_info(sku_id)`	`get_supplier_for_sku()`	supplier (lead time, `allows_expedite`, premium, contact)
`get_demand_velocity(sku_id)`	`get_sales_velocity()`	recent sales rate
`check_active_events(category)`	`get_active_events_for_category()`	events + `events_found` count
`check_capital_budgets(pool_id?)`	`get_capital_pool()` or `get_all_capital_pools()`	pool budget, `auto_approve_limit_aed`, `pool_pressure_flag`

`agents/crew.py`

- **What it does:** Defines a **CrewAI** multi-agent forecasting "crew" (a Senior Data Analyst agent + supporting roles) that Agent 1 calls for a deeper demand read. Uses `llama-3.3-70b-versatile`.
- **Why it matters:** Demonstrates a *second* multi-agent pattern (role-based CrewAI vs graph-based LangGraph) — a strong portfolio point.
- **Role in ORCA:** Sub-crew inside Agent 1's demand analysis.
- **Importance:** ⭐⭐ — **but currently broken** (Issue #1: Groq rejects CrewAI's `cache_breakpoint`; the crew fails and Agent 1 falls back to a raw-data summary). The pipeline still completes via the fallback.

MCP layer

`mcp_server/server.py`

- **What it does:** Runs an **MCP (Model Context Protocol) server** that exposes the six tools so the agent can *discover them dynamically at runtime* instead of importing them as hardcoded functions.
- **Why it matters:** MCP is the same open standard used by Claude Desktop and Cursor. "I exposed my tools via an MCP server" is a current, in-demand interview line.
- **Role in ORCA:** The dynamic tool-discovery layer between the agent and the data tools. The graph connects to it via `langchain-mcp-adapters`.
- **Importance:** ⭐⭐

RAG layer (your evaluation focus)

`docs/*.txt` (the 5 knowledge documents)

- **What it does:** Hold the policy knowledge the database can't express — ordering rules, supplier SLAs, event playbooks, entity relationships, agent reasoning patterns.
- **Why it matters:** They *are* the business logic in prose. **Read all five before writing eval code** — your golden-dataset keywords must match their real wording.

- **Role in ORCA:** The RAG knowledge base (71 chunks once ingested).
- **Importance:** ⭐⭐⭐ (for you specifically)

`docs/rag/ingest.py`

- **What it does:** Reads the 5 docs, splits them into semantic chunks, attaches metadata (`doc_type`), embeds them with the embedding model, and stores them in ChromaDB at `db/chroma/` . Run with `--reset` to rebuild.
- **Why it matters:** No ingest = empty vector store = `RAG: unavailable` . This is the #1 setup gotcha.
- **Role in ORCA:** Builds the searchable knowledge base.
- **Importance:** ⭐⭐⭐

`docs/rag/retriever.py` — your main eval target

- **What it does:** The retrieval brain. Hybrid search (BM25 + vector + RRF), BGE reranking, corrective RAG, metadata filtering. Exposes `query_for_agent1..4()` which each return a **formatted context string**, plus `is_available()` .
- **Why it matters:** This decides *what policy knowledge each agent sees*. Your retrieval eval calls these exact methods.
- **Role in ORCA:** Connects the knowledge base to the agents.
- **Importance:** ⭐⭐⭐ (for you specifically)
- **Note:** Contains the self-healing `is_available()` fix (Issue #11.1 in main KT).

API layer

`api/main.py`

- **What it does:** The FastAPI backend. Key endpoints:

Endpoint	Method	Purpose
`/health`	GET	Liveness + readiness (db, rag, llm, mcp status)
`/api/v1/alerts`	GET	Current critical/at-risk SKU list (dashboard feed)
`/api/v1/pipeline/run`	POST	Launch a pipeline run **as a background task**, returns 202 + pipeline_id immediately
`/api/v1/pipeline/{id}/state`	GET	Current state of a running pipeline (polled every 3s by the dashboard)
`/api/v1/pipeline/{id}/approve`	POST	Human approve/reject → resumes the paused pipeline
`/api/v1/pipeline/{id}/briefing`	GET	The HITL briefing text

`/api/v1/pipelines`	GET	List of pipeline runs (history)

- **Why it matters:** It's the bridge between the UI and the agent pipeline. The **background-task pattern** is crucial: a pipeline takes ~60–90s, so the API kicks it off in the background and returns instantly, then the dashboard polls for progress (otherwise the UI would freeze).
- **Role in ORCA:** The backend server.
- **Importance:** ⭐⭐⭐

`api/models.py`

- **What it does:** Pydantic schemas for every request and response (e.g. `RunPipelineRequest`, `PipelineStateResponse`, `HealthResponse`, plus enums like `PipelineStatus`, `RouteDecision`, `Urgency`).
- **Why it matters:** Guarantees the shape of all data in and out of the API. Bad input → 422 error before any logic runs.
- **Role in ORCA:** The API contract.
- **Importance:** ⭐⭐
- **Note:** `order_qty` is `Optional[float]` on purpose — the LLM sometimes returns decimals like `597.8`, which crashed an earlier `int`-typed version.

Dashboard layer

`dashboard/app.py`

- **What it does:** The Streamlit UI. Three tabs: **Command Centre** (alert list + Analyse buttons + KPI cards), **Pipeline Monitor** (live progress of the running pipeline, auto-refreshing every 3s), **HITL Approval** (Approve/Reject buttons for escalated pipelines).
- **Why it matters:** It's what a human actually sees and clicks. The open-source rebuild of the Palantir Workshop dashboard.
- **Role in ORCA:** The human interface.
- **Importance:** ⭐⭐⭐
- **Note:** Tracks processed SKUs in `session_state` so a SKU's Analyse button becomes ✅ ANALYSED after one click.

`dashboard/api_client.py`

- **What it does:** A thin HTTP wrapper around every API call (`get_alerts()`, `run_pipeline()`, `get_pipeline_state()`, `approve_pipeline()` …). Returns plain dicts; handles errors so the UI never crashes.
- **Why it matters:** Keeps `app.py` pure UI logic — it never touches HTTP directly. Clean separation.
- **Role in ORCA:** UI ↔ API glue.

- **Importance:** ⭐⭐
- **Note:** `BASE_URL` comes from the `API_BASE_URL` env var (so local vs Render just changes one variable).

Eval layer (yours)

`evals/golden_dataset.py`

- **What it does:** 11 hand-written retrieval test cases (real data, real `query_for_agentN` kwargs, expected keywords, forbidden keywords, silent-failure flags).
- **Importance:** ⭐⭐⭐ (your starting point — calibrate it)

`evals/run_retrieval_eval.py`

- **What it does:** Runs the golden cases against the real retriever, scores keyword coverage (recall) and leakage (precision), reports a scorecard, supports `--ci`.
- **Importance:** ⭐⭐⭐ (your Layer 1)

`evals/run_judge_eval.py` ***(to be created by you)*** **— Layer 2**

`.github/workflows/eval_gate.yml` ***(to be created by you)*** **— Layer 3**

Infra / config

File	What it does	Importance
`Dockerfile.api`	Builds the API container (uses slim `requirements.api.txt`)	⭐
`Dockerfile.dashboard`	Builds the dashboard container	⭐
`docker-compose.yml`	Runs API + dashboard together with one command	⭐⭐
`requirements.txt`	Full local dependencies (includes torch, chromadb, sentence-transformers)	⭐⭐
`requirements.api.txt`	Slim deps for Render API (excludes heavy ML libs to fit 512 MB)	⭐⭐
`.env`	Secrets (GROQ_API_KEY etc.) — **never commit**	⭐⭐⭐

PART 2 — End-to-end data flow traces

Now we follow the data. Three scenarios, from the click to the result. Trace #1 is the full walkthrough; #2 and #3 show how the *same* pipeline produces *different* outcomes.

TRACE #1 — "Analyse" → AUTO_EXECUTE (the simplest happy path)

Scenario: A cheap Grocery item, low cost, plenty of budget. No human needed.

The 12 steps

```
[1] USER clicks "Analyse" on SKU00097 (Dish Soap) in Command Centre tab
        │  dashboard/app.py
        ▼
[2] app.py calls api_client.run_pipeline("SKU00097", "STR0001")
        │  dashboard/api_client.py  →  HTTP POST
        ▼
[3] POST /api/v1/pipeline/run  hits  api/main.py
        │  validates body with Pydantic (RunPipelineRequest)
        │  starts the pipeline as a BACKGROUND TASK
        │  returns 202 + {pipeline_id: "PIPE_SKU00097_2026-06-01"} IMMEDIATELY
        ▼
[4] Dashboard switches to Pipeline Monitor tab and starts POLLING
        every 3s:  GET /api/v1/pipeline/PIPE_SKU00097.../state
        (this is why the UI never freezes during the ~60s run)
        ▼
   ── meanwhile, in the background task: agents/graph.py run_pipeline() ──
        ▼
[5] AGENT 1 node (Demand Intelligence)
        ├─ pre-fetches data via tools.py:
        │     check_inventory_positions("SKU00097")  → tools.py → queries.py → orca.db
        │     get_sku_info("SKU00097")               → category="Grocery", abc_class="C"
        │     get_demand_velocity("SKU00097")
        │     check_active_events("Grocery")
        ├─ fetches RAG context:
        │     retriever.query_for_agent1(category="Grocery", abc_class="C",
        │                                urgency=..., lead_time_too_late=...)
        │     → returns a policy/event context STRING
        ├─ (tries CrewAI crew → FAILS on Groq cache_breakpoint → falls back to raw summary)
        ├─ calls LLM (Groq llama-3.1-8b) with prompt + data + RAG context
        └─ writes demand_summary into AgentState
              urgency = MEDIUM,  lead_time_too_late = False
        ▼
[6] AGENT 2 node (Supply Replenishment)
        ├─ get_supplier_info("SKU00097"), get_tier1_stores_for_sku()
        ├─ retriever.query_for_agent2(category="Grocery", supplier_name=..., abc_class="C")
        ├─ LLM builds 3 options (A=standard, B=partial, C=expedite)
        └─ writes options_package into AgentState  (recommended = Option A)
        ▼
[7] AGENT 3 node (Capital Allocation)
        ├─ check_capital_budgets("CP001"), check_capital_budgets("CP003")
        ├─ retriever.query_for_agent3(category="Grocery", urgency="MEDIUM", approval_pool="CP001'
        ├─ LLM scores options with the formula:
        │     budget_score, availability_score, margin_score, lead_time_penalty
        ├─ cost = AED 817   <   CP001 auto_approve_limit (e.g. 50,000)
        └─ writes capital_decision:  approval_required = FALSE
        ▼
[8] ROUTE node (pure Python — NO LLM)
        pool_pressure != HIGH  AND  approval_required == FALSE
```

```
        → route = "AUTO_EXECUTE"
        ▼
[9] HITL interrupt fires (interrupt_before=["execute_node"])
        but for AUTO_EXECUTE the system clears it immediately — no human wait
        ▼
[10] EXECUTE node
        writeback_reorder_for_all_positions("SKU00097")  → queries.py → orca.db
        sets reorder_triggered = "Yes" on all affected positions
        writes an AUTO-EXECUTED confirmation briefing
        ▼
[11] SAVE node
        save_pipeline_run(...)  → db/pipeline_log.py → orca.db
        final_status = AUTO_EXECUTED
        ▼
[12] Dashboard's next 3s poll sees status = AUTO_EXECUTED
        → shows the completed result + briefing. Done. No human action.
```

Key teaching point: Notice steps 5–7 always do the *same two things* — pull **structured data** (tools → queries → DB) and **policy knowledge** (RAG retriever). That double-fetch pattern is the heartbeat of every agent. Your eval measures the RAG half of that heartbeat.

TRACE #2 — "Analyse" → ESCALATE (the HITL path — the important one)

Scenario: Ajwa Dates during Ramadan. Expedite air-freight is needed, and it's expensive enough to need human approval.

Steps 1–7 are the same shape as Trace #1, but the *values* differ:

```
[5] AGENT 1:  9 critical stores + Ramadan event 18 days away,
              supplier lead time 26.75 days > 18 days
              → urgency = CRITICAL,  lead_time_too_late = TRUE
              → RAG context includes Ramadan uplift + expedite rules

[6] AGENT 2:  lead_time_too_late = TRUE  → Option A (standard) marked INSUFFICIENT
              → Option C (expedite) recommended
              (RAG confirms this supplier allows_expedite = Yes)

[7] AGENT 3:  Option C cost = AED 108,909
              CP003 auto_approve_limit = AED 20,000
              108,909 > 20,000  → approval_required = TRUE

[8] ROUTE:    approval_required == TRUE  → route = "ESCALATE"

[9] HITL node (runs BEFORE the interrupt):
        retriever.query_for_agent4(category="Grocery", supplier_name=..., route="ESCALATE")
        LLM writes a BRIEFING: cost, pool, supplier contact, why approval needed
        final_status = ESCALATED
        ▼
   ⏸️  PIPELINE PAUSES HERE  (interrupt_before=["execute_node"])
        the MemorySaver checkpoint stores the full state under thread_id = pipeline_id
        ▼
[10] Dashboard poll sees status = ESCALATED
```

```
        → HITL Approval tab shows the briefing + Approve / Reject buttons
        ▼
    ⏳  ... waits for a HUMAN (could be seconds or hours — state is saved) ...
        ▼
[11] HUMAN clicks Approve
        → app.py → api_client.approve_pipeline(pipeline_id, reviewer)
        → POST /api/v1/pipeline/{id}/approve  →  graph.resume_pipeline(id, approved=True)
        ▼
[12] Pipeline RESUMES from the checkpoint → EXECUTE node runs
        writeback_reorder_for_all_positions(...) → order placed
        SAVE node → final_status = EXECUTED_AFTER_APPROVAL
        ▼
[13] Dashboard poll shows the completed, human-approved result.
```

Key teaching point: The pipeline literally *stops mid-execution* and survives the wait because LangGraph's checkpointer saved every variable. When the human approves hours later, it picks up exactly where it left off. **That pause/resume is the single most important concept in ORCA** — it's the open-source rebuild of Palantir's HITL approval, and it's the answer to the interview question "how do you implement human-in-the-loop?"

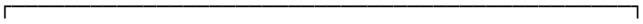
TRACE #3 — "Analyse" → SUSPEND (the budget-blocked path)

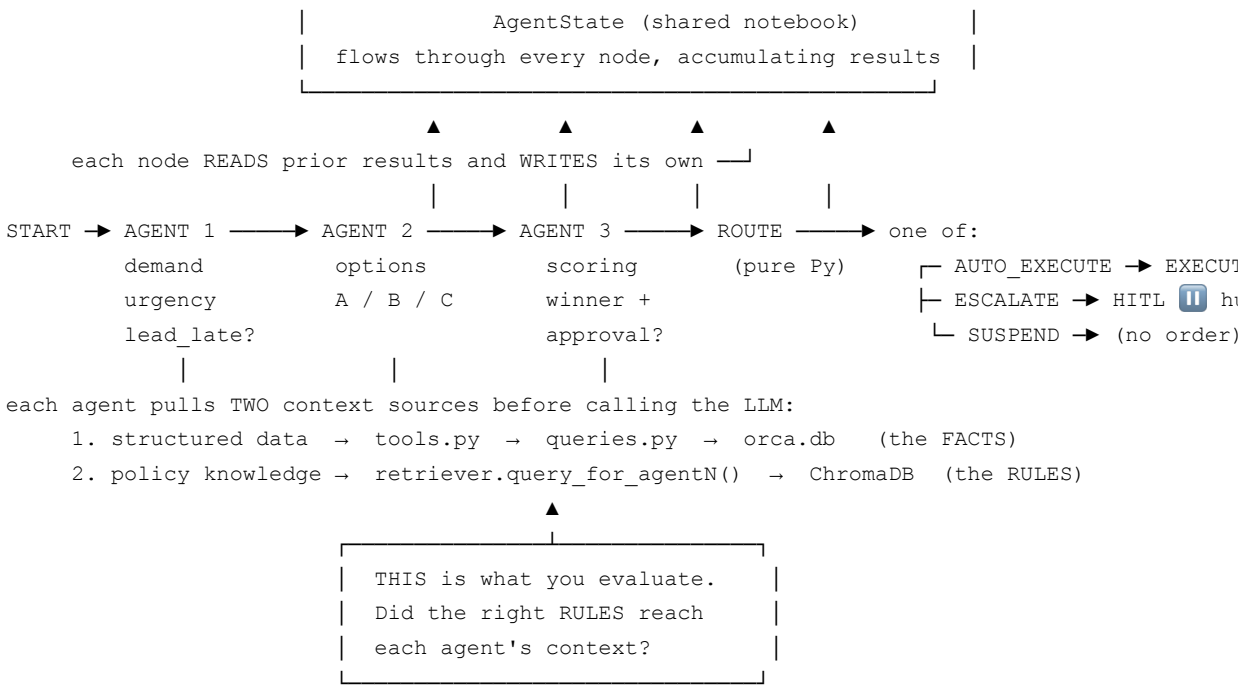
Scenario: A needed order, but the capital pool is out of money this period.

```
[5–7] Agents run normally and produce a recommended option with a cost.

[7] AGENT 3:  the relevant pool's pool_pressure_flag == HIGH
              (budget too tight to take on more commitments now)

[8] ROUTE:    pool_pressure == HIGH  → route = "SUSPEND"
              (this check comes FIRST — it beats both AUTO_EXECUTE and ESCALATE)

[9] SUSPEND node:
        NO order is written. NO money committed.
        Writes a briefing: "SUSPENDED — pool pressure HIGH, order cannot be placed.
        Alert will repeat when pool pressure reduces."
        final_status = SUSPENDED
        ▼
[10] SAVE node → logs it.  Dashboard shows SUSPENDED. No human action, no order.
```

Key teaching point: Routing priority is **SUSPEND first, then AUTO_EXECUTE, then ESCALATE**. A high-pressure pool blocks everything — the system refuses to overcommit a strained budget even for an urgent item. This is a deliberate financial safety rule, and it's exactly the kind of thing your **judge eval** should verify (was SUSPEND chosen correctly when pressure was HIGH?).

The one diagram that ties it all together





How this connects to your eval work

Now that you've seen the flow, your evaluation makes concrete sense:

- **Layer 1 (retrieval eval)** tests the bottom arrow above — for a given agent + situation, does `query_for_agentN()` return a context string containing the right RULES? (e.g. for Trace #2, does Agent 2's context actually contain "expedite" rules and the supplier's expedite permission?)
- **Layer 2 (LLM-as-judge)** tests the *decisions* the agents made with those rules — did Agent 3 apply the scoring formula right? Did ROUTE correctly pick ESCALATE in Trace #2 and SUSPEND in Trace #3? Did Class A never get Option B?
- **Layer 3 (CI gate)** runs Layers 1–2 automatically on every code push, so nobody can silently break a decision.

Re-read Trace #2 once more before you start — the ESCALATE/HITL path is the scenario your judge eval should anchor on first.